CO215, Assignment 8

Roll No : CSB23017

Name : Raktim Ranjan Handique

Session : Spring **Year** : 2025

Objective:

1. To learn about procedures in assembly language
2. To learn passing of parameters (by value, by reference) through stack;
3. To learn writing assembly language procedures.

Exercises: Selection_Sort program

```
CR EQU 0DH
LF EQU 0AH
MAX_SIZE EQU 10
.MODEL SMALL
.STACK 100H
.DATA

array DW 10 DUP (?)
prompt_msg DB 'Enter single digit non-zero integers to be sorted.',
CR, LF
DB 'Enter zero to terminate the input (at most 10).', CR, LF, '$'
output_msg DB 'Array of numbers in sorted order:', CR, LF, '$'
nwln DB CR, LF, '$'
count DW ?
temp DW ?

.CODE

main PROC
.STARTUP

mov ah, 09h
lea dx, prompt_msg
int 21h

mov bx, OFFSET array
mov cx, MAX_SIZE
sub dx, dx

rd_loop:
mov ah, 01h
int 21h
sub al, 30h
mov ah, 0

cmp ax, 0
je stop_read
mov [bx], ax
add bx, 2
inc dx
```

```
loop rd_loop

stop_read:
mov count, dx

call selection_sort

mov ah, 09h
lea dx, output_msg
int 21h

mov cx, count
cmp cx, 0
je exit_program
mov bx, OFFSET array

prt_loop:
mov ah, 02h
mov dx, [bx]
add dl, 30h
int 21h

mov ah, 09h
lea dx, nwln
int 21h

add bx, 2
loop prt_loop

exit_program:
mov ah, 4ch
mov al, 0
int 21h
main ENDP

selection_sort PROC
push ax
push bx
push cx
push dx
push si
push di

mov cx, count
cmp cx, 1
jle sort_done

mov si, 0

outer_loop:
    mov di, si
    mov ax, si
    inc ax

inner_loop:
    cmp ax, cx
```

```
    jge swap_elements

    mov bx, OFFSET array
    mov dx, di
    add dx, dx
    add bx, dx
    mov dx, [bx]

    mov bx, OFFSET array
    push dx
    mov dx, ax
    add dx, dx
    add bx, dx
    mov dx, [bx]
    mov bx, dx
    pop dx

    cmp bx, dx
    jge next_element

    mov di, ax

next_element:
    inc ax
    jmp inner_loop

swap_elements:
    cmp di, si
    je next_iteration

    mov bx, OFFSET array
    mov dx, si
    add dx, dx
    add bx, dx
    mov dx, [bx]
    mov temp, dx

    mov bx, OFFSET array
    mov dx, di
    add dx, dx
    add bx, dx
    mov dx, [bx]

    mov bx, OFFSET array
    push dx
    mov dx, si
    add dx, dx
    add bx, dx
    pop dx
    mov [bx], dx

    mov bx, OFFSET array
    mov dx, di
    add dx, dx
    add bx, dx
    mov dx, temp
```

```
    mov [bx], dx

next_iteration:
    inc si
    cmp si, cx
    jl outer_loop

sort_done:
pop di
pop si
pop dx
pop cx
pop bx
pop ax
ret
selection_sort ENDP

END main
```

Observations:

Bubble_Sort vs Selection_Sort Execution

Bubble_Sort				Selection_Sort		
Instruction No.	**Starting Memory Address**	**Machine Code (Hexadecimal)**		**Instruction No.**	**Starting Memory Address**	**Machine Code (Hexadecimal)**
1	0000	B8 8A 08		1	0000	BAC00B
2	0003	8EDA		2	0003	BEDA
3	0005	8CD3		3	0005	B8CD
4	0007	2BDA		4	0007	2BD2
5	0009	D1E3		5	0009	D1E3
6	000B	D1E3		6	000B	D1E3
7	000D	D1E3		7	000D	D1E3
8	000F	D1E3		8	000F	D1E3
9	0011	FA		9	0011	FA
10	0012	8ED2		10	0012	8ED2
11	0014	03E3		11	0014	03E3
12	0016	FB		12	0016	FB
13	0017	BA09		13	0017	B409
14	0019	BA1500		14	0019	BA1400

15	001C	CD 21		15	001C	CD21
16	001E	BB 0000		16	001E	BB0000
17	0021	B9 0A00		17	0021	B90A00
18	0024	2BD2		18	0024	2BD2
19	0026	B4 01		19	0026	B401
20	0028	CD 21		20	0028	CD21
21	002A	3C 30		21	002A	2C30
22	002C	72 18		22	002C	B400
23	002E	3C 39		23	002E	3D0000
24	0030	77 14		24	0031	740B
25	0032	2C 30		25	0033	8907
26	0034	B4 00		26	0035	83C302
27	0036	3D 0000		27	0038	42
28	0039	74 14		28	0039	E2E8
29	003B	89 07		29	003B	B916A100
30	003D	83 C3 02		30	003F	E82E00
31	0040	42		31	0042	B409
32	0041	E2 E3		32	0044	BA7A00
33	0043	EB 0A		33	0047	CD21
34	0045	90		34	0049	8B0EA100
35	0046	B4 09		35	004D	83F900
36	0048	BA A400		36	0050	7418
37	004B	CD 21		37	0052	BB0000
38	004D	EBD7		38	0055	B402
39	004F	83FA00		39	0057	8B17
40	0052	74 38		40	0059	80C230
41	0054	BB 1416		41	005C	CD21
42	0057	52		42	005E	B409
43	0058	50		43	0060	BA9E00
44	0059	55		44	0063	CD21

45	005A	55		45	0065	83C302
46	005B	8BEC		46	0068	E2EB
47	005D	C746020000		47	006A	B44C
48	0062	5D		48	006C	B000
49	0063	E83300		49	006E	CD21
50	0066	B4 09		50	0070	50
51	0068	BA 7B00		51	0071	53
52	006B	CD 21		52	0072	51
53	006D	BB 0000		53	0073	52
54	0070	B9 0A00		54	0074	56
55	0073	8B0E1400		55	0075	57
56	0077	8B07		56	0076	8B0EA100
57	0079	430		57	007A	83F901
58	007B	8A10		58	007D	7E6E
59	007D	B402		59	007F	BE0000
60	007F	CD21		60	0082	8BFE
61	0081	B402		61	0084	8B4600
62	0083	B220		62	0086	40
63	0085	CD21		63	0087	3BC1
64	0087	83C302		64	0089	7D23
65	008A	E2EB		65	008B	BB0000
66	008C	B409		66	008E	8BBD07
67	0099	55		67	0090	03D2
68	009A	8BEC		68	0092	03DA
69	009C	50		69	0094	8B17
70	009D	53		70	0096	BB0000
71	009E	51		71	0099	52
72	009F	52		72	009A	8BD0
73	00A0	56		73	009C	03D2
74	00A1	57		74	009E	03DA

75	00A2	8B4606		75	00A0	8B17
76	00A5	8B5E04		76	00A2	8BDA
77	00A8	49		77	00A4	5A
78	00A9	7423		78	00A5	3BDA
79	00AB	BB9F00		79	00A7	7D02
80	00AE	BBF300		80	00A9	8BF8
81	00B1	BA0100		81	00AB	40
82	00B2	8B04		82	00AC	EBD9
83	00B4	3B4402		83	00AE	3EFE
84	00B7	7.00E+08		84	00B0	7436
85	00B9	874402		85	00B2	BB0000
86	00BC	8904		86	00B5	8BD6
87	00BE	BA0000		87	00B7	03D2
88	00C1	83C602		88	00B9	03DA
89	00C4	4F		89	00BB	8B17
90	00C5	75EB		90	00BD	8916A300
91	00C7	83FA01		91	00C1	BB0000
92	00CA	7402		92	00C4	8BD7
93	00CC	EBDA		93	00C6	03D2
94	00CE	5F		94	00C8	03DA
95	00CF	5E		95	00CA	8B17
96	00D0	5A		96	00CC	BB0000
97	00D1	59		97	00CF	52
98	00D2	5B		98	00D0	8BD6
99	00D3	58		99	00D2	03D2
100	00D4	5D		100	00D4	03DA
101	00D5	C20400		101	00D6	5A
102	00D8	0000		102	00D7	8917
				103	00D9	BB0000
				104	00DC	8BD7

				105	00DE	03D2
				106	00E0	03DA
				107	00E2	8B16A300
				108	00E6	8917
				109	00E8	46
				110	00E9	3BF1
				111	00EB	7C95
				112	00ED	5F
				113	00EE	5E
				114	00EF	5A
				115	00F0	59
				116	00F1	5B
				117	00F2	58
				118	00F3	C3
				119	00F4	0000

Comparison:

Program	No. of Instructions	Program Length (Bytes)	Number of Memory Operations	Remarks
Bubble Sort	102	216 bytes (approx.)	43	Simple logic with more comparisons and swaps; generally slower but easy to implement.
Selection Sort	119	253 bytes (approx.)	57	Fewer swaps, more memory accesses; better for nearly sorted data; slightly more complex.

Learning Outcome

1. **Parameter Passing:**
    - Understood how values are passed to procedures using push and accessed via the stack using bp+offset.
    - Observed ret 4 for cleaning up the stack after return in bubble_sort.
2. **Use of Alternative Addressing Modes:**
    - Gained experience with indexed addressing (e.g., [bx], [si+2], [bp+6]).
    - Observed differences in pointer manipulation for sorting arrays.
3. **Change of CS and IP Values During Procedure Call and Return:**
    - call instruction pushed the current IP onto the stack.
    - ret (or ret n) popped IP and optionally adjusted SP.
    - Observed flow between main and procedures using debugger.